Back to Math and Physics

A Simple Time-Corrected Verlet Integration Method

Programming | Math and Physics

Published February 07, 2005 by Jonathan "lonesock" Dummer, posted by Myopic Rhino

Do you see issues with this article? Let us know.

ADVERTISEMENT

Introduction

Verlet integration is a nifty method for numerically integrating the equations of motion (typically linear, though you can use the same idea for rotational). It is a finite difference method that's popular with the Molecular Dynamics people. Actually, it comes in three flavors: the basic Position, the Leapfrog and the Velocity versions. We will be discussing the Position Verlet algorithm in this paper. It has the benefit of being quite stable, especially in the face of enforced boundary conditions (there is no explicit velocity term with which the position term can get out of sync). It is also very fast to compute (almost as fast as Euler integration), and under the right conditions it is 4th order accurate (by comparison, the Euler method is only 1st order accurate, and the second order Runge-Kutta method is only 2nd order accurate [go figure]).

The disadvantages of the Verlet method are that it handles changing time steps badly, it is not a self-starter (it requires 2 steps to get going, so initial conditions are crucial), and it is unclear from the formulation how it handles changing accelerations. In this paper we will discuss all of these shortcomings, and see how to minimize their impact. The modified Verlet integrator is referred to as the Time-Corrected Verlet (TCV) and is shown below with its original counterpart. The computations used to generate the graphs for this paper are included in this Excel file.

Original Verlet:

$$x_{i+1} = x_i + (x_i - x_{i-1}) + a * dt * dt$$

Time-Corrected Verlet:

$$x_{i+1} = x_i + (x_i - x_{i-1)} * (dt_i / dt_{i-1}) + a * dt_i * dt_i$$

To see why the TCV is an improvement, we need to see the math behind the original Verlet method.

Math

In this paper we will be talking exclusively about point masses, which are acted on by forces. Well, we all know about Newton's little equation: Force=d(Momentum)/dt. Momentum=mass*velocity and for non-relativistic speeds the mass is constant, removing our need for the chain rule, and yielding our familiar F=ma. So if all the forces acting on the point mass are summed (the vector F), then scaled by (1.0/m) we have the acceleration (a) of the point mass. Since we know how to go from force to acceleration, we will be starting from the acceleration term to keep the math less cluttered. In actual applications you will almost always be summing forces, then converting to accelerations.

Most of the graphs presented in this paper include the matching Euler simulation, just for reference. The Euler algorithm is extremely simple, and it will not be derived here. However, the equations are included below for your reference (order is important):

$$v = v + a * dt$$
$$x = x + v * dt$$

There is a more accurate version, but it is not strictly the Euler method, so it will not be used for this paper. However it does give 2nd order accurate results, instead of merely 1st order:

$$x = x + v * dt + 0.5 * a * dt * dt$$
$$v = v + a * dt$$

Please note that there is a faster way to derive the Verlet method's math* than what will be shown here, however it does not provide the insight needed to overcome the issues mentioned in the introduction. So we'll start from a few basic principles: a=dv/dt, and v=dx/dt. So for any given point mass, if we know the current position, velocity and acceleration (the acceleration must be constant over the given time step), we can compute exactly where it will be after the time step (dt) has elapsed.

(1) $$x_{i+1} = x_i + v_i * dt_i + 0.5 * a_i * dt_i * dt_i$$

or

(1a) $x_{i+1} - x_i = v_i * dt_i + 0.5 * a_i * dt_i * dt_i$

Now, if we wanted that equation formulated without the velocity term, we could replace it with some other known state variable, such as the position x. But since we don't know x_{i+1} yet, we shift the whole equation back a step:

(2) $x_i - x_{i-1} = v_{i-1} * dt_{i-1} + 0.5 * a_{i-1} * dt_{i-1} * dt_{i-1}$

and we can use simple integration to see that:

(3) $v_i = v_{i-1} + a_{i-1} * dt_{i-1}$

or

(3a) $v_{i-1} = v_i - a_{i-1} * dt_{i-1}$

and we substitute that into equation (2):

(4) $x_i - x_{i-1} = (v_i - a_{i-1} * dt_{i-1}) * dt_{i-1} + 0.5 * a_{i-1} * dt_{i-1} * dt_{i-1}$

or

(4a) $x_i - x_{i-1} = v_i * dt_{i-1} - 0.5 * a_{i-1} * dt_{i-1} * dt_{i-1}$

For this next step we need to make a big assumption, the importance of which will be seen later: If we assume that neither the acceleration nor the time step vary between steps (i.e. that $a_{i-1} = a_i = a$ and that $dt_{i-1} = dt_i = dt$) then we note that:

(5) $x_i - x_{i-1} + a * dt * dt = v_i * dt - 0.5 * a * dt * dt + a * dt * dt$

or

(5a) $x_i - x_{i-1} + a * dt * dt = v_i * dt + 0.5 * a * dt * dt$

You'll notice that the right hand side of equation (5a) is exactly the last half of equation (1), so we can work the modified equation (5a) back into equation (1):

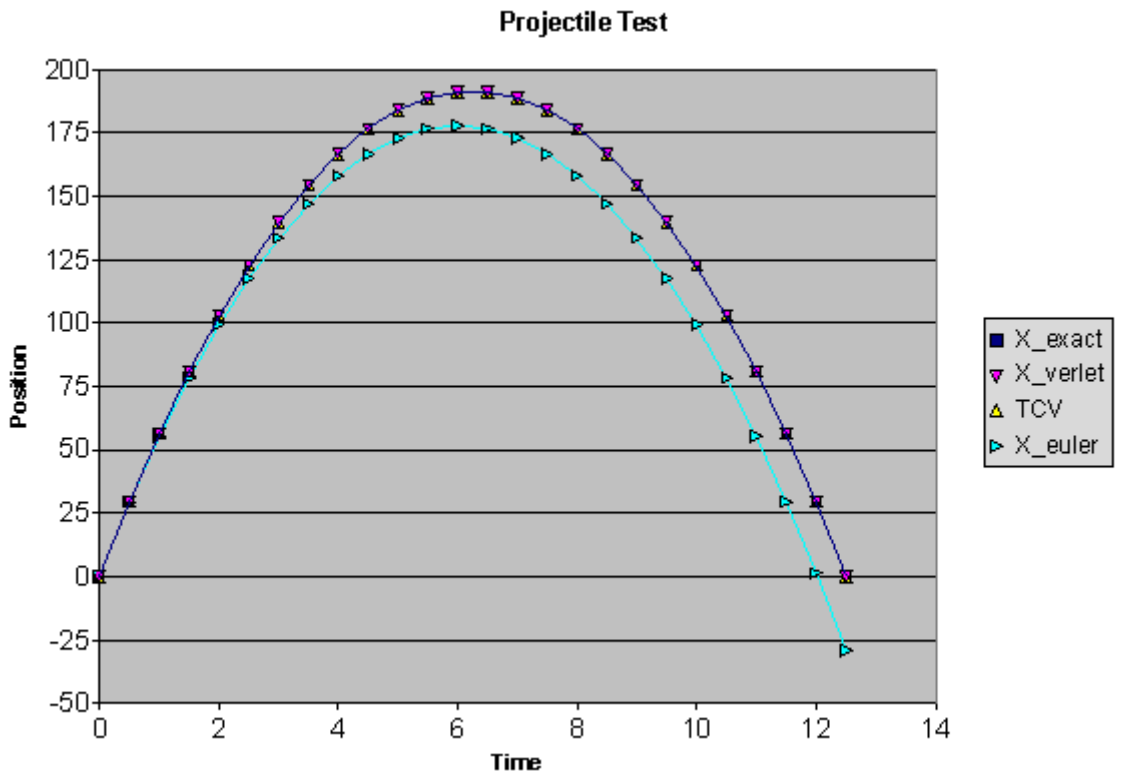
(6) $x_{i+1} = x_i + (x_i - x_{i-1}) + a * dt * dt$

and there you have the traditional Verlet Position integration method.

Fundamental Problems

As you saw from the derivation's step (5), the two criteria needed to make the Verlet algorithm exact are constant acceleration and constant time step. For most practical cases we cannot guarantee either of these criteria. Of course, there are some simple cases where both criteria will be met. For example, simple projectile physics simulated by a physics engine which uses fixed time steps will yield perfect results. As soon as you add friction, springs or constraints of any kind you nullify the constant acceleration criterion, and adapting your time step to your game's framerate will nullify the constant time step criterion.

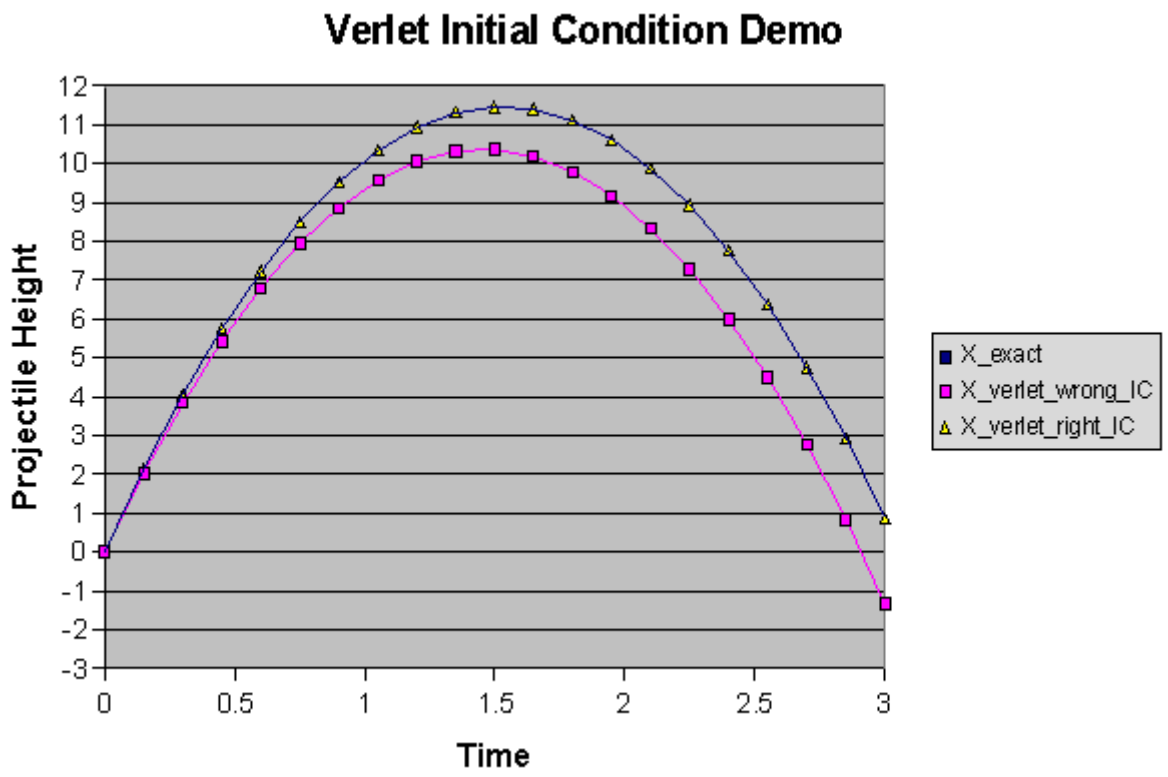
I am not going to address the constant acceleration criterion, mainly because explicit integrators (such as this one) must assume the constant acceleration principle, which is violated the instant you start simulating a complex system. Take the example of a point mass connected to a spring: as soon as it starts to move, the spring force, and thus the acceleration, changes. Even equation (1) required that the acceleration be constant throughout a time step. The error introduced by assuming that $a_{i-1} = a_i$ is actually implicit in our choosing an explicit scheme without knowing how the acceleration changes. We are effectively setting d(a)/dt (a.k.a. the "jerk") to 0.0. The other reason I will ignore this issue is that, empirically, the standard Verlet method already handles changing accelerations better than the Euler method (or even the improved variation on the Euler method), as long as the time step is fixed. Note that the Time-Corrected Verlet will be identical to the original Verlet when the time step is fixed. Observe the following graphs:



The error introduced by the constant time step assumption is something that can be easily ameliorated. This will be shown in the section entitled "A Simple Time-Correction Scheme".

Implementation Problems

A large source of inaccuracy when using the Verlet scheme stems from the improper specification of initial conditions. Looking at equation (6) and trying to fit it into the form of equation (1) (which may be more familiar) may yield (improper) reasoning like this: the first (x_i) term is the position contribution, the second (x_i - x_{i-1}) term is basically the velocity contribution, and the (a * dt * dt) term is clearly the acceleration contribution. So when simulating the traditional projectile path, setting x_0 = 0.0, and $x_{-1}=x_0$ - v_0 * dt_0, and running the simulation from there will give the wrong results, as can be seen in the following graph:



This is because both the second and third terms include acceleration information. So, when setting the current state explicitly (i.e. the position and velocity initial conditions), remember to use equation (2). Shooting simulations depend upon starting with accurate initial conditions. The larger the initial time step, the less accurate your computed initial state will be if you do not use equation (2).

A Simple Time-Correction Scheme

The remaining fundamental problem with the Verlet integration method lies with its assumption of a constant time step. The $(x_i - x_{i-1})$ term from equation (4a) is the portion of the equation which is dependent on the constant acceleration and constant time step assumptions. It has been explained (OK, hand-waved away) why the usage of the last time step's acceleration will not be corrected, but we still have the problem of dt_{i-1} being stale information. Rewriting equation (4) yet again, we see that:

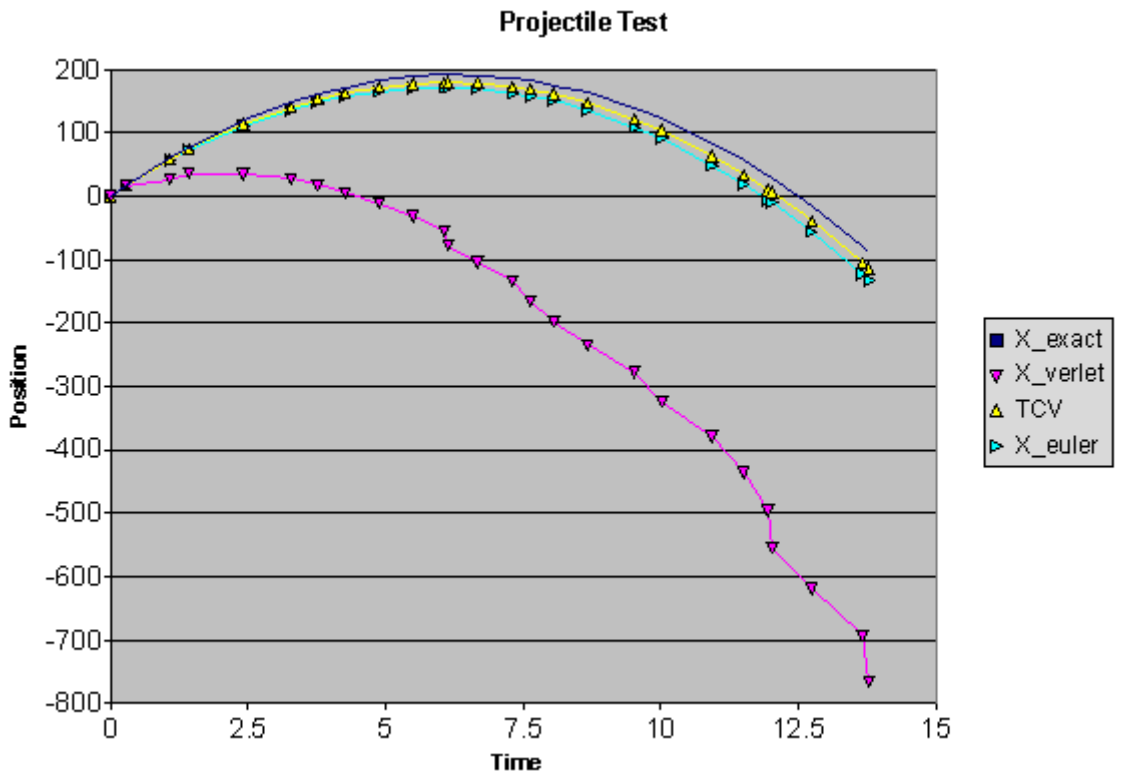
(4b) $x_i - x_{i-1} = (v_i - 0.5 * a_{i-1} * dt_{i-1}) * dt_{i-1}$

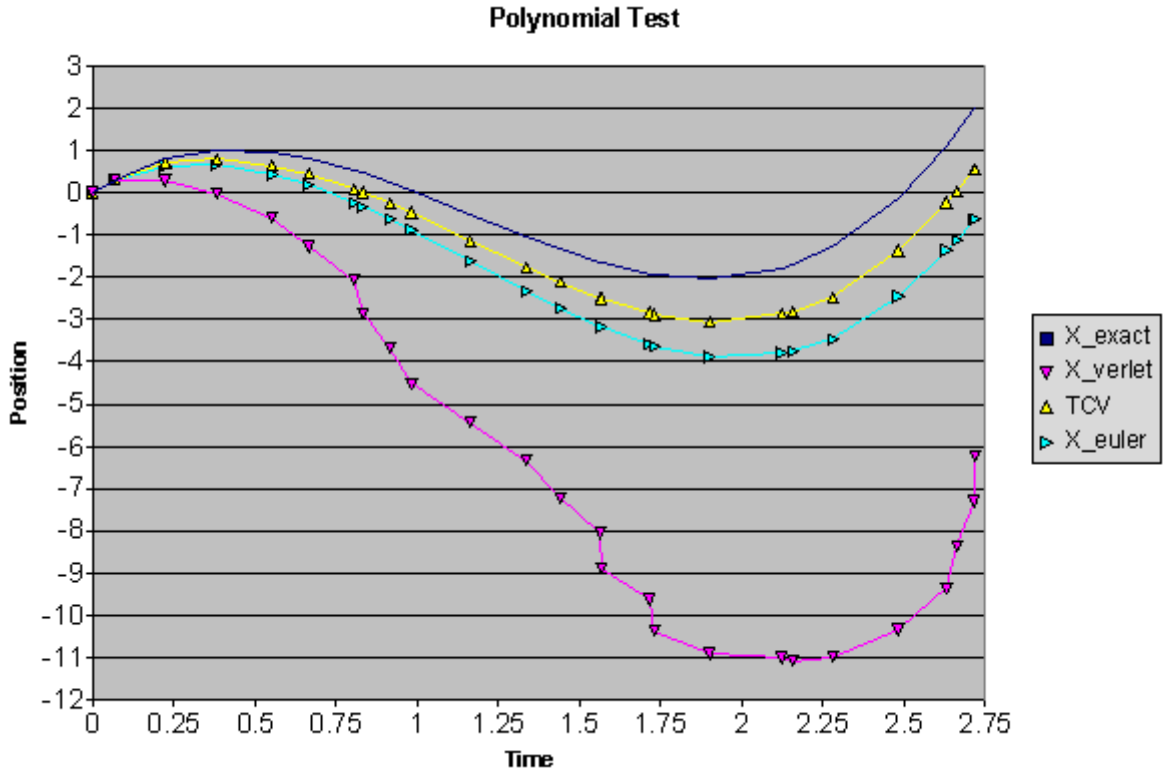
so taking the easy way out and ignoring the dt_{i-1} linked with the acceleration, I can swap out the old dt_{i-1} for my new dt_i by multiplying (4b) by (dt_i / dt_{i-1}). Plugging all of this in yields my final form of the Time-Corrected Verlet integration method:

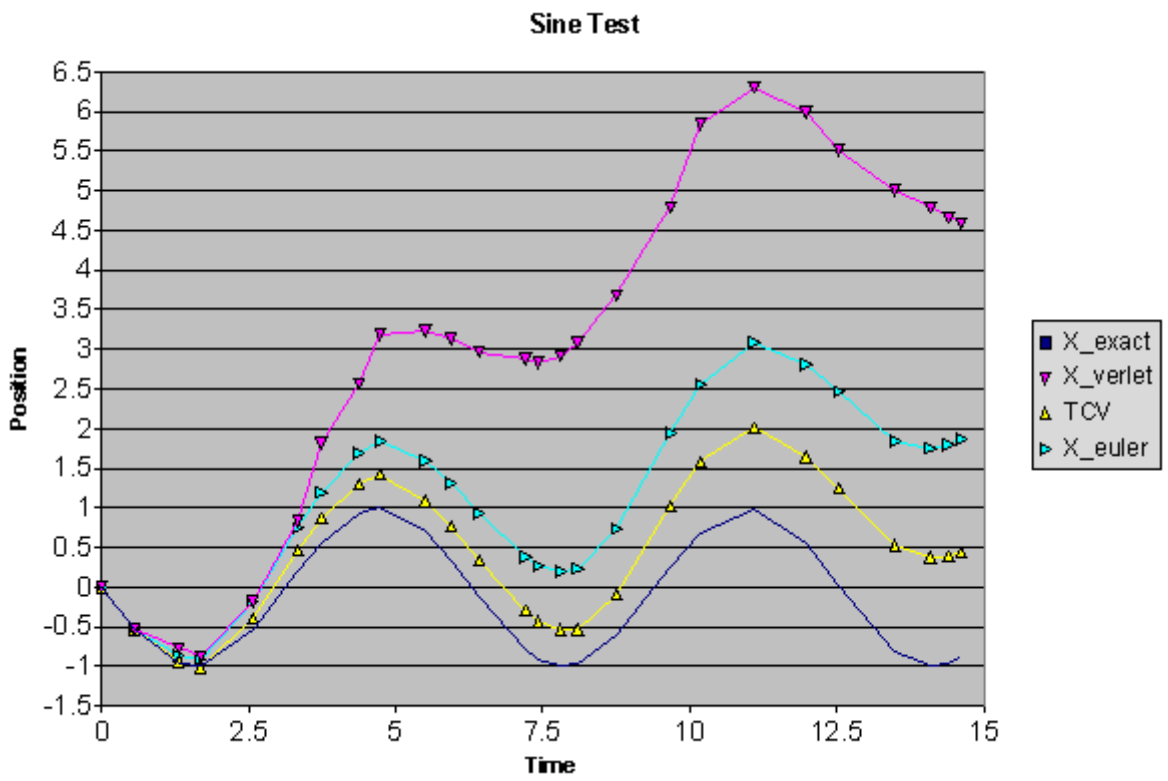
(7) $x_{i+1} = x_i + (x_i - x_{i-1)} * (dt_i / dt_{i-1}) + a * dt_i * dt_i$

You will notice that when the last frame's time step equals the current frame's time step, the modifier term becomes 1.0, yielding the traditional form of the Verlet equation. As an optimization note, only one value needs to be stored per frame, as the dt was constant for all points in the last frame. So the term (dt_i / dt_{i-1}) can be computed once per frame and stored in a variable. At the end of the Verlet update subroutine simply store the current frame's dt as old_dt. Likewise, $dt_i * dt_i$ can be computed once and stored in a variable, saving a multiplication.

To show how the Time-Corrected Verlet behaves, a spreadsheet was set up with the TCV, the original Verlet and Euler's method, each simulating three different problems with known solutions. These are the same tests as were performed earlier, but with randomized time steps. Of course since every time step (except the first) was random, the graph looked different each time a simulation was "run". Sometimes both the original Verlet and the TCV simulations were similar, however the original Verlet always fell further away from the exact solution than the TCV version eventually. Here are some sample graphs:







Conclusion

Using the Time-Corrected form of the Verlet integration method with the proper equation for initializing the state makes the TCV integration scheme a simple, yet powerful method for doing game physics, even with changing frame rates.

It restores some of the accuracy of the method (still 4th order with constant time steps, between 2nd and 4th with changing dt), while maintaining its cheap numerical cost. I hope this helps a bit when implementing your own physics simulation code.

* Hint: remember the central difference 2nd derivative approximation?

$d^2x / dt^2 = a_i^2 = (x_{i+1} - 2*x_i + x_{i+1}) / dt^2$

0 Likes 0 Comments Share:

LATEST COMMENTS

Nobody has left a comment. You can be the first!

You must log in to join the discussion.
Don't have an account? Sign up!

ADVERTISEMENT